



High Performance Computing Exercise Sheet 5

HS 25
Dr. Douglas Potter

<http://www.ics.uzh.ch/>

Teaching Assistants:

Cesare Cozza, cesare.cozza@uzh.ch

Mark Eberlein, mark.eberlein@uzh.ch

Issued: 17.10.2025

Due: 24.10.2025

This exercise session focuses on coding Poisson solver and OpenMP parallelization. You can find the provided code in the git repository in the folder `exercise_session_05`.

Exercise 1 [Serial 2D Poisson solver]

In this exercise, you will (partially) implement a serial 2D Poisson solver based on the Jacobi method, while in the next exercise you will parallelize it. In order to guide you, we prepared a skeleton of the code that you will have to finalize.

The equation

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f(x, y) \text{ with } x, y \in [0, 1] \times [0, 1] = \mathcal{D}, \quad (1)$$

defines a Poisson problem together with boundary conditions: $u = u_B$ on $\partial\mathcal{D}$ (anywhere x or y is 0 or 1). We consider two Poisson problems:

$$f(x, y) = 8\pi^2 \sin(2\pi x) \sin(2\pi y), \quad u_B \equiv 0 \quad (2)$$

and

$$f(x, y) = 0, \quad u_B = \begin{cases} 1 & \text{if } 0.25 < x < 0.75 \text{ and } y = 0 \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

One possible way of solving this problem numerically is to successively apply Jacobi iterations and converge to an approximate solution. The domain $\mathcal{D}$ is split into a uniform grid with N^2 elements of width $\Delta = L/(N-1) = 1/(N-1)$. Let's start the procedure with taking a first guess for the solution $u^{(0)}$, that is the source term (f) everywhere in $\mathcal{D}$. This can be written $u_{i,j}^{(0)} = f_{i,j}$ for any $i, j \in [0, N-1]$, where $u_{i,j}^{(0)} = u^{(0)}(x = i\Delta, y = j\Delta)$. The Jacobi iteration:

$$u_{i,j}^{(k+1)} = \frac{1}{4} \left(u_{i-1,j}^{(k)} + u_{i+1,j}^{(k)} + u_{i,j-1}^{(k)} + u_{i,j+1}^{(k)} - \Delta^2 f_{i,j} \right), \text{ with } i, j \in [1, N-2] \quad (4)$$

will progressively update the $u^{(k)}$ to a closer approximation of the solution of Eq. 1.

The procedure should stop when $\|u^{(k)} - u^{(k-1)}\| = \sqrt{\frac{\sum_{ij} (u_{ij}^{(k)} - u_{ij}^{(k-1)})^2}{N^2}} < \text{tol}$, where tol is the tolerance, which can be chosen arbitrarily.

In the course repository `exercise_session_05/poisson_solver`, you will find the code `poisson_solver`, in which the routines `init_f`, `jacobi_step` and `norm_diff` are missing.

- What are these routines supposed to do?

Write down the missing routines and test your implementation of the code. You should obtain one output file every 1000 Jacobi iteration. These files can be visualized using the python script `output/plot.py`.

Commit: Commit your implementation and a visualization of your results.

(the above description is valid in case of the same amount of grid points along both x and y axis. One can also generalize this to cases with different amount of grid points along x and y , respectively)

Exercise 2 [OpenMP Poisson solver] In this exercise, you will have to parallelize the Jacobi Poisson solver you coded in the last exercise with `OpenMP`. Copy your serial code to a new directory and parallelize the code in the new directory. Obviously, the most time consuming operation is updating the grid components for each jacobi step, then comes the estimate of difference and error. Starting from the serial version of the code, then:

- 1) Update the Makefile in order to compile with the `OpenMP` library.
- 2) Insert `OpenMP` instructions at the appropriate spots to parallelize the `jacobi_step` and the `norm_diff` computations.
- 3) Submit jobs with 1 and 4 threads and make sure that you get same answers.

Commit: Push your OMP parallel codes into git repository.

Exercise 3 [OpenMP and race conditions]

In this exercise you will parallelize with `OpenMP` a simple code which computes the maximum value and the number of 0 in a long array of integers (see file `num.txt`). You can find a serial

version of the code in the git repository `exercise_session_05/race_conditions`. Remember that, by default, all variables in the `#pragma omp parallel` regions are `shared`. Therefore pay particular attention to which variables you want to keep `shared` and which ones you want to make `private`.

- First task, parallelize the first loop which computes the maximum value of the array. Use a simple `#pragma omp parallel for` region and keep the same structure. Run it multiple times with different number of threads and check whether it always outputs the correct result.
- Now parallelize in the same way also the second loop, counting the occurrences of 0 in the array. Run it again multiple times. What happens?
- **Commit:** If you weren't cautious enough in the precedent tasks, you probably run into a race condition. One way to solve it is to use the `#pragma omp atomic` or `#pragma omp critical` directives. Where is it best to insert these directives in order to avoid a serialization of your code? Another way to parallelize the two loops is to use `#pragma omp reduction` clauses. Try this option.
- **Commit:** Check (make sure) that your code also compiles in serial.